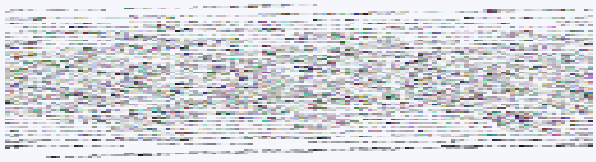
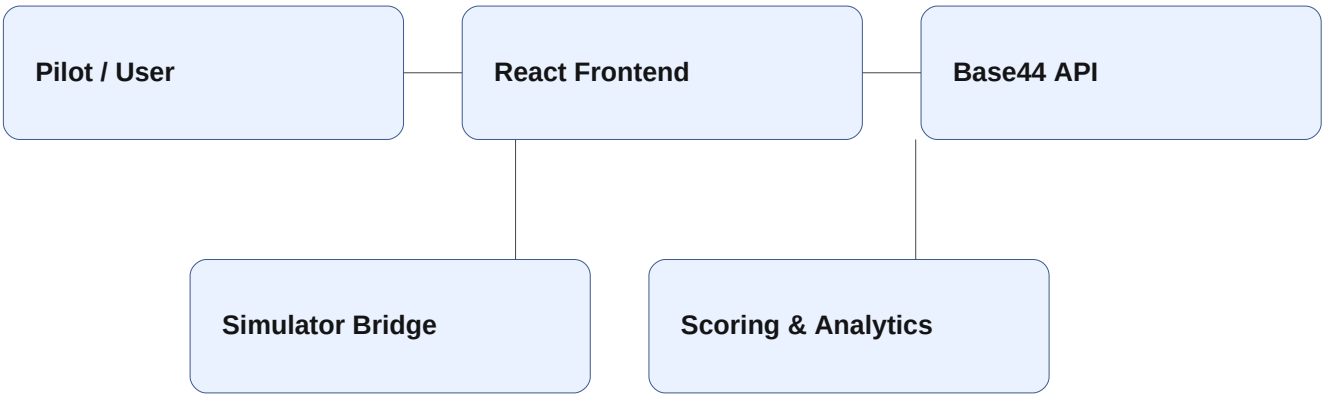


SkyCareerV1 Manual

English Simple Extended

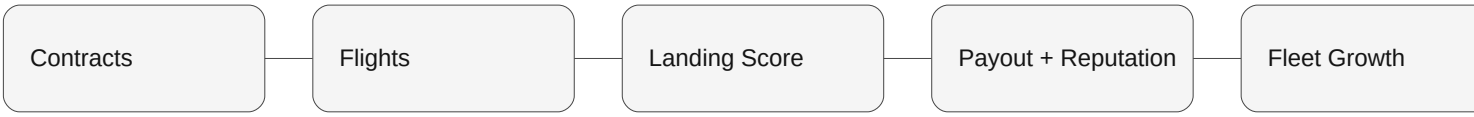
Generated automatically from docs/SkyCareerV1_Manual_EN.md

Figure 1: SkyCareerV1 System Architecture



This architecture diagram shows the basic data and control flow between users, frontend UI, backend entities, simulator bridge tools, and analytics modules.

Figure 2: Player Progress Loop



Improvement in SkyCareerV1 is cyclical: better operations improve your money and reputation, which unlocks better contracts and stronger aircraft opportunities.

SkyCareerV1 Operations Manual (English)

Version date: 2026-05-04

Goal: Practical, simple, and very detailed reference manual for users/admins/support.

1. Executive Summary

SkyCareerV1 combines airline business simulation with flight-sim mission execution. You plan contracts, execute flights, manage fleet and finances, and progress through ratings and achievements.

2. Quickstart

1. Step 1: Complete setup, choose contract, fly, debrief, improve.
2. Step 2: Complete setup, choose contract, fly, debrief, improve.
3. Step 3: Complete setup, choose contract, fly, debrief, improve.
4. Step 4: Complete setup, choose contract, fly, debrief, improve.

5. Step 5: Complete setup, choose contract, fly, debrief, improve.

6. Step 6: Complete setup, choose contract, fly, debrief, improve.

7. Step 7: Complete setup, choose contract, fly, debrief, improve.

8. Step 8: Complete setup, choose contract, fly, debrief, improve.

9. Step 9: Complete setup, choose contract, fly, debrief, improve.

10. Step 10: Complete setup, choose contract, fly, debrief, improve.

11. Step 11: Complete setup, choose contract, fly, debrief, improve.

12. Step 12: Complete setup, choose contract, fly, debrief, improve.

3. Functional Tour by Page

1. CompletedFlightDetails

What this page is for: CompletedFlightDetails is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

2. ActiveFlights

What this page is for: ActiveFlights is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

3. PerformanceCalculator

What this page is for: PerformanceCalculator is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

4. About

What this page is for: About is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

5. Contracts

What this page is for: Contracts is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

6. Fleet

What this page is for: Fleet is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

7. AdminAircraftImages

What this page is for: AdminAircraftImages is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

8. Achievements

What this page is for: Achievements is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

9. FlightTracker

What this page is for: FlightTracker is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

10. FlightHistory

What this page is for: FlightHistory is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

11. ContractDetails

What this page is for: ContractDetails is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

12. XPlaneDebug

What this page is for: XPlaneDebug is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

13. XPlaneSetup

What this page is for: XPlaneSetup is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

14. Setup

What this page is for: Setup is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

15. Finances

What this page is for: Finances is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

16. GameSettingsAdmin

What this page is for: GameSettingsAdmin is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

17. Landing

What this page is for: Landing is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

18. Account

What this page is for: Account is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

19. FreeFlight

What this page is for: FreeFlight is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

20. Contact

What this page is for: Contact is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

21. TypeRatings

What this page is for: TypeRatings is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

22. Dashboard

What this page is for: Dashboard is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

23. FlightMap

What this page is for: FlightMap is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

24. Leaderboard

What this page is for: Leaderboard is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

25. AdminDiscounts

What this page is for: AdminDiscounts is part of the product workflow and should be used whenever this domain is active.

What to check first: account state, simulator connection, aircraft readiness, and financial viability.

Typical actions: review data cards, confirm mission/asset conditions, execute action buttons, and monitor result messages.

Success criteria: no blocking errors, valid operational outcome, and synchronized company progression metrics.

Common mistakes: skipping preflight assumptions, poor aircraft-contract matching, and incomplete post-flight review.

Operator tip: always compare expected reward versus operational risk before continuing.

4. Components Deep Dive

Component 1: LevelBonusInfo (finance)

Path: `src/components/finance/LevelBonusInfo.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 2: CreditInfoCard (finance)

Path: `src/components/finance/CreditInfoCard.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 3: SkyDollarPacks (finance)

Path: `src/components/finance/SkyDollarPacks.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 4: ComparisonSection (landing)

Path: `src/components/landing/ComparisonSection.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 5: demoData (landing)

Path: `src/components/landing/demoData.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 6: AppScreenshot (landing)

Path: `src/components/landing/AppScreenshot.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 7: PricingSection (landing)

Path: `src/components/landing/PricingSection.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 8: LangToggle (landing)

Path: `src/components/landing/LangToggle.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 9: SignupGate (landing)

Path: `src/components/landing/SignupGate.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 10: translations (landing)

Path: `src/components/landing/translations.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 11: NewFeaturesShowcase (landing)

Path: `src/components/landing/NewFeaturesShowcase.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 12: LeaderboardRow (leaderboard)

Path: `src/components/leaderboard/LeaderboardRow.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 13: LeaderboardFilters (leaderboard)

Path: `src/components/leaderboard/LeaderboardFilters.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 14: LeaderboardHeader (leaderboard)

Path: `src/components/leaderboard/LeaderboardHeader.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 15: TypeRatingMissionPopup (typerating)

Path: `src/components/typerating/TypeRatingMissionPopup.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 16: UserNotRegisteredError (UserNotRegisteredError.jsx)

Path: `src/components/UserNotRegisteredError.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 17: RealMoneyBuyButton (store)

Path: `src/components/store/RealMoneyBuyButton.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 18: DeleteAccountDialog (account)

Path: `src/components/account/DeleteAccountDialog.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 19: InsolvencyBanner (InsolvencyBanner.jsx)

Path: `src/components/InsolvencyBanner.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 20: translations (i18n)

Path: `src/components/i18n/translations.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 21: CreateDiscountForm (admin)

Path: `src/components/admin/CreateDiscountForm.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 22: DiscountList (admin)

Path: `src/components/admin/DiscountList.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 23: PricingCards (subscription)

Path: `src/components/subscription/PricingCards.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 24: SubscriptionPaywall (subscription)

Path: `src/components/subscription/SubscriptionPaywall.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 25: SubscriptionStatus (subscription)

Path: `src/components/subscription/SubscriptionStatus.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 26: ActiveFlightCard (flights)

Path: `src/components/flights/ActiveFlightCard.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 27: failureUtils (flights)

Path: `src/components/flights/failureUtils.js`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 28: LandingQualityVisual (flights)

Path: `src/components/flights/LandingQualityVisual.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 29: FuelPrediction (flights)

Path: `src/components/flights/FuelPrediction.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 30: aircraftModels3D (flights)

Path: `src/components/flights/aircraftModels3D.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 31: glbScenery (flights)

Path: `src/components/flights/glbScenery.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 32: centerlineColor (flights)

Path: `src/components/flights/centerlineColor.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 33: WeatherDisplay (flights)

Path: `src/components/flights/WeatherDisplay.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 34: FinalApproach3D (flights)

Path: `src/components/flights/FinalApproach3D.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 35: SimBriefImport (flights)

Path: `src/components/flights/SimBriefImport.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 36: glbLoader (flights)

Path: `src/components/flights/glbLoader.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 37: approachGeometry (flights)

Path: `src/components/flights/approachGeometry.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 38: PreFlightCalculator (flights)

Path: `src/components/flights/PreFlightCalculator.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 39: NearbyAirports (flights)

Path: `src/components/flights/NearbyAirports.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 40: customAircraftModel (flights)

Path: `src/components/flights/customAircraftModel.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 41: RouteWaypoints (flights)

Path: `src/components/flights/RouteWaypoints.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 42: realisticTrees (flights)

Path: `src/components/flights/realisticTrees.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 43: useReplayRecorder (flights)

Path: `src/components/flights/useReplayRecorder.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 44: aircraftModelCatalog (flights)

Path: `src/components/flights/aircraftModelCatalog.js`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 45: FlightMapIframe (flights)

Path: `src/components/flights/FlightMapIframe.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 46: airportDetails (flights)

Path: `src/components/flights/airportDetails.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 47: runwayAccuracy (flights)

Path: `src/components/flights/runwayAccuracy.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 48: customAirportModel (flights)

Path: `src/components/flights/customAirportModel.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 49: aircraftSpeedLookup (flights)

Path: `src/components/flights/aircraftSpeedLookup.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 50: cityBuildings (flights)

Path: `src/components/flights/cityBuildings.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 51: RunwayAccuracyCard (flights)

Path: `src/components/flights/RunwayAccuracyCard.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 52: LandingScoreCalculator (flights)

Path: `src/components/flights/LandingScoreCalculator.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 53: TakeoffLandingCalculator (flights)

Path: `src/components/flights/TakeoffLandingCalculator.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 54: AdvancedLandingScore (flights)

Path: `src/components/flights/AdvancedLandingScore.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 55: FlightProfileChart (flights)

Path: `src/components/flights/FlightProfileChart.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 56: RouteCorridorAirports (flights)

Path: `src/components/flights/RouteCorridorAirports.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 57: landingMetrics (flights)

Path: `src/components/flights/landingMetrics.js`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 58: landingMetrics (flights)

Path: `src/components/flights/landingMetrics.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 59: airportScene (flights)

Path: `src/components/flights/airportScene.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 60: FlightMap (flights)

Path: `src/components/flights/FlightMap.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 61: roadNetwork (flights)

Path: `src/components/flights/roadNetwork.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 62: ActiveFailuresDisplay (flights)

Path: `src/components/flights/ActiveFailuresDisplay.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 63: useMp4Exporter (flights)

Path: `src/components/flights/useMp4Exporter.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 64: generatePassengerComments (flights)

Path: `src/components/flights/generatePassengerComments.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 65: landingBonusRecovery (flights)

Path: `src/components/flights/landingBonusRecovery.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 66: FlightRating (flights)

Path: `src/components/flights/FlightRating.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 67: FlightCompletionAnimation (flights)

Path: `src/components/flights/FlightCompletionAnimation.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 68: ProtectedRoute (ProtectedRoute.jsx)

Path: `src/components/ProtectedRoute.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 69: achievementDefinitions (achievements)

Path: `src/components/achievements/achievementDefinitions.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 70: achievementProgress (achievements)

Path: `src/components/achievements/achievementProgress.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 71: AchievementUnlockList (achievements)

Path: `src/components/achievements/AchievementUnlockList.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 72: achievementRewards (achievements)

Path: `src/components/achievements/achievementRewards.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 73: processAchievementsAfterFlight (achievements)

Path: `src/components/achievements/processAchievementsAfterFlight.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 74: AchievementsWidget (achievements)

Path: `src/components/achievements/AchievementsWidget.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 75: useAchievements (achievements)

Path: `src/components/achievements/useAchievements.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 76: achievementEvaluator (achievements)

Path: `src/components/achievements/achievementEvaluator.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 77: AchievementBadge (achievements)

Path: `src/components/achievements/AchievementBadge.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 78: HotspotInfoPopup (fleet3d)

Path: `src/components/fleet3d/HotspotInfoPopup.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 79: MaintenanceCategoryList (fleet3d)

Path: `src/components/fleet3d/MaintenanceCategoryList.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 80: maintenanceHotspots (fleet3d)

Path: `src/components/fleet3d/maintenanceHotspots.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 81: AircraftHangar3D (fleet3d)

Path: `src/components/fleet3d/AircraftHangar3D.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 82: Fleet3DView (fleet3d)

Path: `src/components/fleet3d/Fleet3DView.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 83: MarketHangar3DView (fleet3d)

Path: `src/components/fleet3d/MarketHangar3DView.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 84: hangarScene (fleet3d)

Path: `src/components/fleet3d/hangarScene.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 85: ContractCard (contracts)

Path: `src/components/contracts/ContractCard.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 86: HangarModelPreview3D (contracts)

Path: `src/components/contracts/HangarModelPreview3D.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 87: ContractWorldMap (contracts)

Path: `src/components/contracts/ContractWorldMap.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 88: FuelCalculator (contracts)

Path: `src/components/contracts/FuelCalculator.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 89: HangarMarket3D (contracts)

Path: `src/components/contracts/HangarMarket3D.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 90: hangarModelCatalog (contracts)

Path: `src/components/contracts/hangarModelCatalog.js`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 91: HangarWorldGlobe3D (contracts)

Path: `src/components/contracts/HangarWorldGlobe3D.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 92: hangarModelCatalog (contracts)

Path: `src/components/contracts/hangarModelCatalog.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 93: MaintenanceCategories (aircraft)

Path: `src/components/aircraft/MaintenanceCategories.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 94: AircraftCard (aircraft)

Path: `src/components/aircraft/AircraftCard.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 95: XPlaneStatus (dashboard)

Path: `src/components/dashboard/XPlaneStatus.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 96: CreditScoreBadge (dashboard)

Path: `src/components/dashboard/CreditScoreBadge.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 97: ReputationGauge (dashboard)

Path: `src/components/dashboard/ReputationGauge.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 98: StatCard (dashboard)

Path: `src/components/dashboard/StatCard.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

Component 99: LanguageContext (LanguageContext.jsx)

Path: `src/components/LanguageContext.jsx`

Purpose (simple): This component helps users understand, control, or visualize a specific feature in the app.

Input/Output pattern: receives route/user/mission data, renders status/controls, emits user actions and computed UI outcomes.

Support usage: if users report confusion in this area, verify data availability and expected view state transitions.

Operational note: when this view fails, validate authentication state, query loading, and entity consistency.

5. Data and Domain Model

Entity 1: Subscription

Path: `base44/entities/Subscription.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 2: Flight

Path: `base44/entities/Flight.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 3: LemonCatalogItem

Path: `base44/entities/LemonCatalogItem.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 4: Contract

Path: `base44/entities/Contract.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 5: Hangar

Path: `base44/entities/Hangar.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 6: Company

Path: `base44/entities/Company.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 7: Aircraft

Path: `base44/entities/Aircraft.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 8: GameSettings

Path: `base44/entities/GameSettings.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 9: XPlaneLog

Path: `base44/entities/XPlaneLog.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

Entity 10: AircraftTemplate

Path: `base44/entities/AircraftTemplate.jsonc`

Business role: stores persistent career data used across contracts, flights, payouts, and progression logic.

Risk if inconsistent: mission corruption, payout mismatch, or incomplete history/analytics.

6. Supporting Libraries and Service Files

Support File 1

Path: `src/utils/index.ts`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 2

Path: `src/utils/airportCoordinates.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 3

Path: `src/utils/additionalAirportCoordinates.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 4

Path: `src/api/base44Client.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 5

Path: `src/lib/AuthContext.jsx`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 6

Path: `src/lib/typeRatings.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 7

Path: `src/lib/maintenance.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 8

Path: `src/lib/app-params.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 9

Path: `src/lib/insurance.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 10

Path: `src/lib/lemonItemCatalog.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 11

Path: `src/lib/utils.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 12

Path: `src/lib/NavigationTracker.jsx`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 13

Path: `src/lib/payoutFactors.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 14

Path: `src/lib/PageNotFound.jsx`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

Support File 15

Path: `src/lib/query-client.js`

Why it matters: this file contributes integration, utility, state, or platform behavior that keeps the product stable.

When to inspect: setup failures, routing issues, auth errors, map inconsistencies, scoring anomalies, or API mismatch.

7. 20 End-to-End Scenarios

Scenario 1

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 2

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 3

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 4

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 5

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 6

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 7

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 8

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 9

1. Validate user/account state.
2. Validate simulator/bridge signal.
3. Choose or continue operation.
4. Monitor KPI and warnings.
5. Confirm mission result, payout, and progression updates.
6. Record lessons learned for next mission cycle.

Scenario 10

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 11

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 12

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 13

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 14

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 15

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 16

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 17

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 18

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 19

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

Scenario 20

1. Validate user/account state.

2. Validate simulator/bridge signal.

3. Choose or continue operation.

4. Monitor KPI and warnings.

5. Confirm mission result, payout, and progression updates.

6. Record lessons learned for next mission cycle.

8. Troubleshooting Catalog

Troubleshooting Case 1

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 2

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 3

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 4

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 5

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 6

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 7

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 8

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 9

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 10

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 11

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 12

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 13

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 14

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 15

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 16

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 17

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.

- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 18

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 19

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 20

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 21

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 22

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 23

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 24

- Symptom: user sees inconsistent status, missing score, or blocked progression.

- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 25

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 26

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 27

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 28

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 29

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 30

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 31

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 32

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 33

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 34

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 35

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 36

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 37

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 38

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 39

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

Troubleshooting Case 40

- Symptom: user sees inconsistent status, missing score, or blocked progression.
- Root causes: stale integration state, incomplete mission prerequisites, or entity mismatch.
- Resolution: re-check setup path, rerun mission prechecks, and verify post-flight persistence.

9. Training Curriculum

Lesson 1

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 2

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 3

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 4

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.

- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 5

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 6

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 7

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 8

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 9

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 10

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 11

- Objective: improve operational discipline and score reliability.

- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 12

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 13

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 14

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 15

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 16

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 17

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 18

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 19

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 20

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 21

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 22

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 23

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

Lesson 24

- Objective: improve operational discipline and score reliability.
- Exercise: run a constrained mission and compare expected vs actual result.
- Debrief: identify one decision that improved safety, timing, or economy.

10. Appendix: Command Reference

- Generate PDF: `node scripts/generate_manual_pdf.mjs`
- Keep this manual as the source of truth for onboarding, QA walkthroughs, and support runbooks.

11. Extended Support Playbooks

Playbook 1

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 2

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 3

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 4

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 5

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 6

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 7

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 8

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 9

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 10

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 11

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 12

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 13

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 14

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 15

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 16

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 17

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 18

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 19

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 20

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 21

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 22

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 23

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 24

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 25

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 26

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 27

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 28

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 29

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 30

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 31

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 32

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 33

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 34

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 35

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 36

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 37

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 38

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 39

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 40

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 41

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 42

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 43

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 44

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 45

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 46

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 47

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 48

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 49

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 50

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 51

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 52

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 53

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 54

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 55

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 56

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 57

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 58

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 59

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 60

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 61

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 62

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 63

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 64

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 65

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 66

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 67

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 68

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 69

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 70

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 71

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 72

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 73

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 74

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 75

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 76

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 77

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 78

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.
- Mitigate: safe fallback workflow with minimal business loss.
- Confirm: user can continue progression loop without data loss.

Playbook 79

- Intake: collect route, aircraft, simulator, timestamp, and page context.
- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.

Playbook 80

- Intake: collect route, aircraft, simulator, timestamp, and page context.

- Verify: contract prerequisites, active flight state, and final debrief persistence.

- Mitigate: safe fallback workflow with minimal business loss.

- Confirm: user can continue progression loop without data loss.